Frontend Web Development Standart Stack For PT.LSKK

v1.0 — 20 Juli 2026. Dipecah dari dokumen Development Standart Stack (FE & BE) tanpa perubahan substansi aturan; kode yang sebelumnya berupa gambar ditranskrip menjadi teks. Bagian dari keluarga standar LSKK: Frontend Web, Backend, dan Mobile.

1. Mandatory Rules

- Lang : Typescript -> https://www.typescriptlang.org/
- Library : React JS (Functional Components) -> https://react.dev/
- Module Bundler : Vite -> https://vite.dev/
- CSS Lib : Tailwind CSS -> https://tailwindcss.com/
- UI Components : Ant-Design -> https://ant.design/
- Table : React Data Grid -> https://mui.com/x/react-data-grid/
- Data Fetching : Axios -> https://axios-http.com/docs/intro
- Data Query Library : Tanstack Query -> https://tanstack.com/
- Global State Management : Context, Zustand -> https://zustand-demo.pmnd.rs/
- Router : React Router Dom (CreateHashRouter) -> https://reactrouter.com/
- Prettier Active Extensions : -> https://prettier.io/
- Linter : Default By Vite
- Form Validation : React Hook Form -> https://react-hook-form.com/
- Schema Validation : Zod -> https://zod.dev/
- Mobile First Approach -> Responsive
- Must use data types, interfaces and enums. Minimize the use of any data types.

2. Response Wrapper Interface Data Fetching

Generic Response Wrapper:

```
export interface IResponseEntity<T> {
  code: number;
  status: boolean;
  message: string;
  data?: T;
  meta?: ImetaPagination;
}
```

Meta for pagination interface:

```
export interface ImetaPagination {
  totalPages: number;
  totalData: number;
  totalDataPerPage: number;
  page: number;
  limit: number;
}
```

Example Response:

```
{
  "code": 200,
  "status": true,
  "message": "get all case success",
  "data": [],
  "meta": {
    "totalPages": 0,
    "totalData": 0,
    "totalDataPerPage": 0,
    "page": 1,
    "limit": 10
  }
}
```

3. Lazyloading (Performance)

```
const UseCallBackPage = lazy(() => import("../pages/UseCallBackPage"));

const routers: ReturnType<typeof createBrowserRouter> = createBrowserRouter([
  {
    element: <MemoPage />,
    path: "/",
  },
  {
    element: <UseCallBackPage />,
    path: "/use-callback",
  },
]);
```

4. Memoize (Performance)

The term "memoize" refers to optimizing web performance by preventing unnecessary re-renders of child components when the props they depend on in the parent components change.